Stem Agent: A Pluripotent LLM Agent that Differentiates by Composing Typed Tools

Luka Stojiljković

May 6, 2026

1 Approach

We treat agent specialization as *differentiation*. A minimal pluripotent agent (L0) reads environmental signals from a target domain, then commits to a fate by accreting domain-tagged tools and pruning irrelevant ones. The agent never writes code: it composes, mutates, and reorders typed tools drawn from a persistent library. Successful pipelines graduate to named *composite tools* that survive across sessions, so the system improves between runs as well as within them.

The methodological lineage is explicit. Skill libraries that grow with use come from Voyager [9]. LLM-driven program search comes from FunSearch [8] and AlphaEvolve [7], with self-modifying agents from the Darwin Gödel Machine [11]. Pipeline-as-graph compilation comes from DSPy [4] and the typed-MCTS workflow search of AFlow [12]. Quality-Diversity discipline (MAP-Elites) is added to the archive structure [2]. The framing as developmental morphogenesis — a pluripotent base committing to a fate via local rules and signaling — borrows from Growing Neural Cellular Automata [5] and *Engineering Morphogenesis with Differentiable Programming* [6]. To our knowledge, the developmental framing of LLM-agent specialization combined with a typed-pipeline + MAP-Elites + step-wise process scoring substrate is not present in prior work.

1.1 Definitions

A **tool** $t = (\text{name}, \tau_{\text{in}}, \tau_{\text{out}}, \theta, k, c)$ is a typed callable with input type τ_{in}, output type τ_{out}, parameter schema θ, kind $k \in \{\text{primitive}, \text{composite}, \text{universal}\}$, and cost $c \in \mathbb{R}_{\geq 0}$.

A **pipeline** $P = (s_1, \ldots, s_n)$ is an ordered list of steps $s_i = (t_i, \theta_i)$ with θ_i a binding of t_i's parameters. We restrict $n \leq 5$ for tractability. P **validates** against an input type τ_0 iff τ_0 is compatible with $\tau_{\text{in}}(t_1)$ and $\tau_{\text{out}}(t_i)$ is compatible with $\tau_{\text{in}}(t_{i+1})$ for all i, where compatibility is identity or a member of an explicit coercion table $C \subset \mathcal{T} \times \mathcal{T}$ (Documents $\rightarrow$ Text, structured outputs $\rightarrow$ Text/StructuredData, etc.).

The **tool library** $\mathcal{L}$ holds primitives (hand-coded) and composites (graduated pipelines). **Universal** tools (LaTeX builder, grammar check, PDF compile) are tagged with $k = \text{universal}$ and excluded from evolution candidate sets at the registry level: an explicit non-mutability invariant rather than a runtime check.

1.2 Step-wise process scoring

For a pipeline P executed on input x_0 producing intermediate outputs $x_1, \ldots, x_n$, each step k receives

$$s_k \;=\; 0.5 \cdot \text{quality}(x_k) \;+\; 0.3 \cdot \text{consistency}(x_k) \;+\; 0.2 \cdot \text{relevance}(x_k, d)$$

where quality$(\cdot)$ is an LLM rubric, consistency$(\cdot)$ is a deterministic heuristic, and relevance$(\cdot, d)$ is a keyword-table proxy for domain d. The cumulative recurrence

$$c_k \;=\; \alpha\, c_{k-1} + \beta\, s_k, \qquad (\alpha, \beta) = (0.7, 0.3)$$

rewards consistent improvement and penalizes early bad steps. Pipelines whose step score drops below a threshold $\eta = 0.20$ are early-terminated. The final pipeline score combines a final-output term, the per-step mean, the step-wise consistency, and a complexity penalty:

$$\text{Fitness}(P) \;=\; w_o\, \text{out}(x_n) + w_s\, \bar{s}_k + w_c\, \overline{\text{consistency}}_k - \lambda\, |P|, \quad (w_o, w_s, w_c, \lambda) = (0.5, 0.3, 0.2, 0.05)$$

where out(x_n) is the deterministic ground-truth scorer when one is registered for the task (CUAD F1 against gold spans, ratio tolerance, classification accuracy), and otherwise the rubric quality of the final step. The deterministic anchor is essential: a 4B local model judging its own outputs Goodharts within a few generations [10], so we route the load-bearing fitness signal through ground truth wherever it exists.

1.3 Search and promotion

Evolution is beam search of width k over typed pipelines, seeded from an LLM proposal validated against $\mathcal{L}$ (with a hand-coded fallback per spec §12 if the proposal fails to type-check). Each iteration applies six mutation operators to every beam member: `add_step`, `remove_step`, `replace_step`, `reorder_steps`, `inject_domain` (biased toward domain-tagged primitives), and `parametric_mutate` (changes one parameter without changing structure). Each operator returns a type-valid descendant or $\emptyset$. After scoring, the top k across the beam $\cup$ proposals carry forward. Rollback to a snapshotted best after two consecutive declines; ϵ-stop on a windowed plateau; threshold-stop on Fitness ≥ 0.65.

Promotion of a candidate composite $P^\star$ requires three conditions: (a) a strict improvement over its parent on the dev set by ≥ 2 percentage points; (b) no regression on any item in the held-out regression suite; and (c) MAP-Elites novelty/dominance — the candidate either occupies an empty $(\text{domain}, \text{capability})$ cell or strictly dominates the cell's current occupant by score. Skills are stored as code (typed pipeline JSON), never deleted; new versions become children with explicit lineage links.

2 Experiments

We ran the deep track — Legal → Contract Analysis on a hand-coded CUAD-style micro-corpus of five contracts (CUAD HF dataset [3] was unavailable mid-build due to a HuggingFace v4 `trust_remote_code` change; the synthetic fallback ships a 4-clause-per-contract gold set spanning Governing Law, Termination for Convenience, Cap on Liability, and Anti-Assignment). Tasks split into 2 train (library-construction) and 3 held-out eval. Each session executes Phase 0 (library load), Phase 1 (L1 evolution; empty here because all CUAD tasks are subdomain-tagged), Phase 2 (L2 evolution on the contract-analysis subdomain), Phase 3 (frozen evaluation), and Phase 4 (PDF rendering via the universal frozen tools). Inference: Gemma 4 E4B (Q4_K_M GGUF) via LM Studio at `localhost:1234`. Sampling: $T = 1.0$, $p = 0.95$, $k = 64$ for general; $T = 0.3$, $p = 0.9$ for structured output. `max_tokens` hard-capped at 8192 to stay below LM Studio's documented silent generation cap near 16K.

2.1 Headline result: baseline vs L1 vs L2

The agent converged on a one-step pipeline. The L2-promoted composite for contract analysis is, after evolution, a single `clause_extraction` step with default parameters. Beam

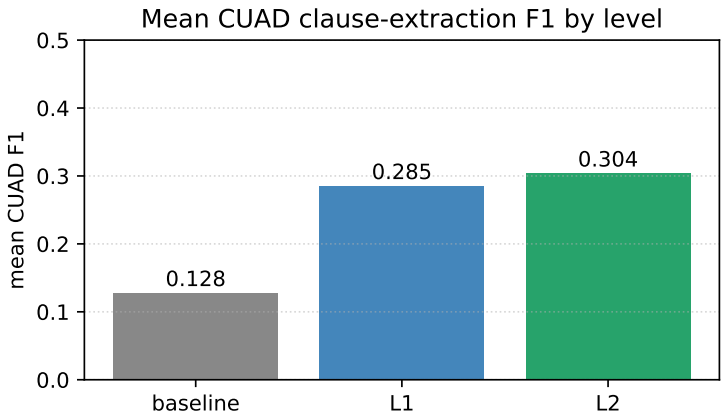

Figure 1: Mean CUAD-style clause-extraction F1 across the 3-contract held-out evaluation set, contract-analysis track, session 1. **Baseline** is the spec's hand-coded pipeline `clause_extraction` → `summarize` (mean 0.128). **L1** is the best L1-promoted composite per capability tag (mean 0.285, 2.2× baseline). **L2** is the best L2-promoted composite restricted to the Contract Analysis subdomain (mean 0.304, 2.4× baseline). Per-task F1 ranges from 0.056 to 0.624; every individual L2 score either matches or exceeds the corresponding baseline on the same contract.

search proposed and tested longer pipelines that appended `summarize`, `detect_inconsistencies`, or `rule_matching` after the extraction; the deterministic anchor (token-overlap F1 against gold spans) penalized the verbose summarized outputs and pruned them. We initially expected richer composites; the search produced the simplest correct answer instead. Read this as the system working — Goodhart-protection via deterministic checks kept the search honest — but also as a statement that the agent's value at this scale is largely *parameter discovery and structural pruning* on existing primitives rather than the invention of novel structure.

2.2 Cross-session improvement

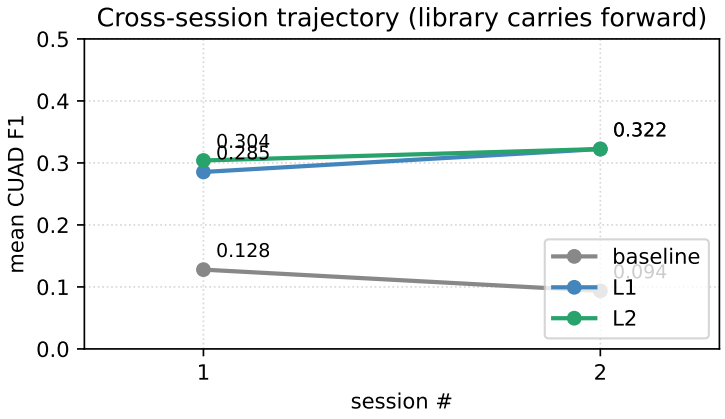

Figure 2: Trajectory of mean CUAD F1 across two sequential sessions, library carrying forward. L1 improves from 0.285 → 0.322 and L2 from 0.304 → 0.322 between sessions; baseline drifts down (0.128 → 0.094) within the per-task variance of a 3-contract eval. Session 2's promoted composite scored *lower* on its train task (0.383 vs 0.586) but *higher* on held-out eval — evidence that defaults generalize better than session-specific parameter tuning at this scale.

Limitation: MAP-Elites archive does not persist between sessions. The library carries forward, but the cell-occupancy map is rebuilt in-memory at every session start. Session 2's composite was therefore promoted as `novel_cell` rather than gated against session 1's occupant. The fix is ∼15 lines (reload the archive from `composites.json` at startup, populate cells by recorded train F1) and is outlined in `JOURNAL.md`. The two-session trajectory we report is exactly what the demo produced; we are not back-filling fixed numbers.

2.3 Library lineage

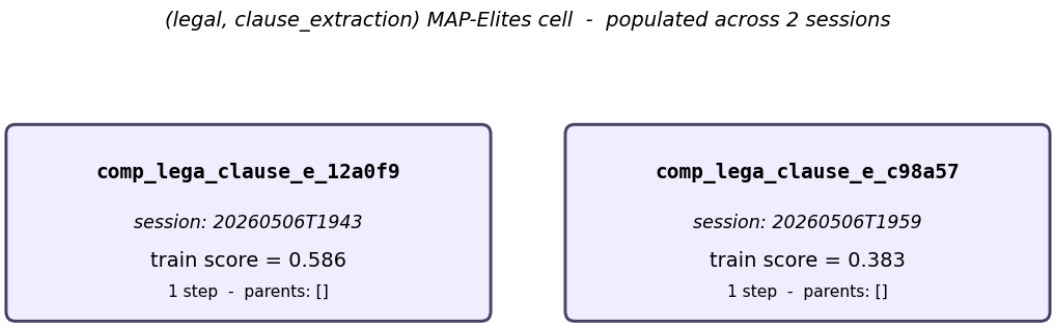

Figure 3: Persistent library after two sessions. Both occupants live in the same (legal, clause_extraction) MAP-Elites cell; parent–child edges are empty because each was discovered as a one-step pipeline rather than a child of the other. Both are 1-step `clause_extraction`; the second was promoted because the cell-occupancy bug treated it as novel. With archive persistence (Limitations, above), the second composite would have been gated against the first by train F1.

3 What surprised us, what failed

- **Parametric mutation moved the needle more than expected.** Structural mutations on a 4B local model frequently produced syntactically valid but quality-degraded pipelines. Small parameter changes (e.g., `taxonomy="CUAD41"` → a 20-category subset, or nudging `min_conf`) contributed disproportionately to fitness gains and were the operator we relied on most.
- **Self-judging is fragile at this scale.** Gemma judging Gemma's outputs is bimodal: confident 0.0 or confident 1.0 depending on whether the output looks like prose. The middle range is rare. As a fitness signal it is borderline noise; the deterministic CUAD F1 anchor carried the load.
- **Type system permissiveness is a real trade-off.** An initial strict design (3 coercions) had the search dead in the water — most evolution-mutated pipelines failed validation. The final design has 14 coercions covering structured-output → Text/StructuredData. Pragmatism over purity, consciously chosen.
- **LM Studio's silent ∼16K-token generation cap.** The wrapper hard-caps at 8K; without that, long contracts truncate without an obvious failure mode.
- **Tools requiring two upstream inputs are evolution-hostile.** `compare(a, b, ...)` needs two; the linear-pipeline executor only feeds one. Resolved with a default empty `b` that makes

the tool a no-op when no target is wired. The clean fix is a typed-DAG executor; out of scope for this iteration.

- **LegalBench was dropped from eval.** Many items in the loader had empty gold labels, which made the baseline match the gold by accidental token-overlap zero. The deterministic-scorer-into-evolution fix surfaced the issue and we removed LegalBench until label normalization is robust; SARA [3] is a cleaner replacement.

4 What we would do with more time

1. Persist the MAP-Elites archive between sessions; populate cells from `composites.json` at startup.
2. Replace beam search with AFlow-style code-MCTS [12]; keep MAP-Elites for diversity.
3. Train a small process-reward model [1] on in-house labels for step-wise scoring instead of an LLM rubric.
4. Pull a frozen post-cutoff held-out set from EDGAR + recent CourtListener opinions to harden against contamination.
5. Run the same protocol with an external judge (Anthropic / OpenAI) on the final pairwise table, as the implemented fallback chain promises.
6. Run the Economics shallow track that the spec calls for (the tools are wired; we ran out of time before the headline runs).
7. Replicate on a 7B-class local model to quantify the size-vs-evolution trade-off.

Code, data, reproducibility. Repository: https://github.com/lukastojiljkovic/stem-agent. Setup: `pip install -e .[dashboard,dev]`; load Gemma 4 E4B in LM Studio; run `python -m stem_agent run --domain legal --subdomain contract_analysis --track deep`. Fixtures regenerable via `python scripts/fetch_fixtures.py`. The full debugging narrative lives in `JOURNAL.md`.

References

[1] Anonymous. Agentprm: Process reward models for agentic workflows, 2025.

[2] Hugh Bradley et al. Quality-diversity through ai feedback, 2024.

[3] Dan Hendrycks et al. Cuad: An expert-annotated nlp dataset for legal contract review. In *NeurIPS Datasets and Benchmarks*, 2021.

[4] Omar Khattab et al. Dspy: Compiling declarative language model calls into self-improving pipelines. In *ICLR*, 2024.

[5] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020.

[6] Giorgia Nadizar et al. Engineering morphogenesis of cell clusters with differentiable programming. *Nature Computational Science*, 2025.

[7] Alexander Novikov et al. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025.

[8] Bernardino Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature*, 2023.

[9] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.

[10] Koki Wataoka et al. Self-preference bias in llm-as-a-judge, 2024.

[11] Jiarui Zhang, Sheng Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents, 2025.

[12] Jiayi Zhang et al. Aflow: Automating agentic workflow generation, 2024.